

05 · Serielle Bussysteme

Mikrocomputertechnik 3 · Die Big 3 — Wie Mikrocontroller mit der Welt sprechen

Prof. Dr.-Ing. Daniel Klünder

Folie

1 / 80

Willkommen zu Block 5!

- Willkommen zur fünften Vorlesung in Mikrocomputertechnik 3
- Bisher: Programmierung und Hardware weitgehend *innerhalb* des Chips
- Auch PWM war vor allem Energie an einem Ausgangs-Pin
- Heute: physikalische Grenze des SoCs überschreiten
- Ziel: komplexe Daten mit externen Chips austauschen (Text, Messwerte, Konfiguration)

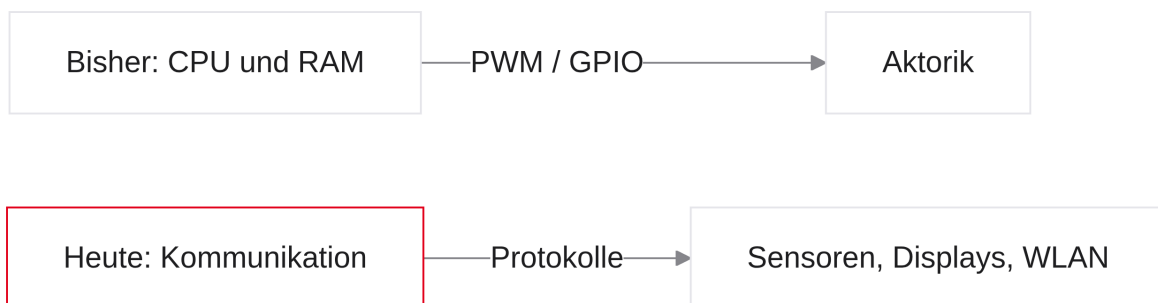
Notizen

Herzlich willkommen zurück! Speicher, Interrupts und Timer beherrschen Sie bereits. Moderne Systeme brauchen aber Sensoren, Displays und WLAN-Module. Wie transportieren wir Bits sicher über Kupferbahnen außerhalb des SoCs? Das ist das Thema heute.

Folie

2 / 80

Einordnung im Semester



Notizen

Von interner Steuerung zu On-Board-Kommunikation mit „smarten“ Peripheriebausteinen.

Anknüpfung: PWM und Watchdog

Kurz-Recap aus Block 4 (Vertiefung im Übungsblatt):

- Hardware-PWM: ARR → Frequenz, CCR → Duty Cycle
- Signal läuft jitterarm in der Peripherie, CPU orchestriert
- Watchdog: Freeze erkennen, harter Reset als Selbsthilfe
- Leitprinzip: harte Pin-Arbeit in Hardware, nicht per Bit-Banging

```
void main_loop(void) {  
    fade_led();  
    wdt_feed();  
}
```

Notizen

Behalten Sie bei: Die CPU orchestriert; Echtzeit-Pinarbeit gehört in dedizierte Module. Heute gilt dasselbe für Kommunikations-Hardware (UART/SPI/I2C-Controller).

Die SoC-Grenze

- Beispiel: digitaler Beschleunigungssensor (IMU) auf der Platine
- Der Sensor speichert Beschleunigungsrohwerter (X, Y, Z), hier als 16-Bit-Zweierkomplement
- Neigung ist daraus nur unter Bewegungsannahmen abgeleitet, kein Registerinhalt
- Frage: Wie kommen diese Bits in die Register/ALU unseres SoCs?
- Chips sind durch Kupferbahnen getrennt
- Wir brauchen ein physikalisches und zeitliches Regelwerk — ein Protokoll

Notizen

Innerhalb der CPU verschieben wir Worte über breite interne Busse. Zwischen zwei Chips zählen Laufzeiten, Störungen und gemeinsame Regeln.

SoC und Sensor — Schema

Vom Sensorregister zur CPU. Drei Adressarten, drei Verdrahtungen.

Lehrschem. GND gehoert dazu, zaehlt aber nicht als Signalleitung.

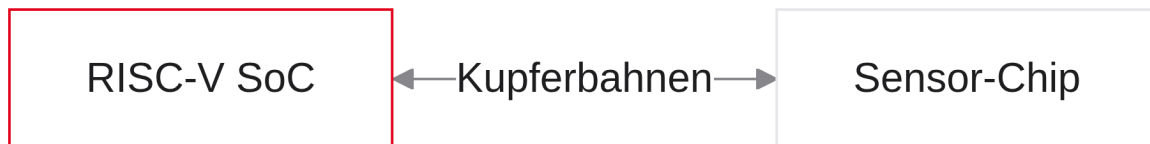


Getrennte Symbole: Quadrat = SoC-MMIO, Sechseck nur als Text: I2C-Adresse 0x68, gestrichelt = internes Register.



MPU6050-Beispiel: Beschleunigungswerte. Neigung ist daraus nur unter Annahmen abgeleitet.

Abbildung 1: Drei Adressen: MMIO im SoC, I²C-Geräteadresse 0x68, Sensorregister ab 0x3B.



Notizen

Schematisch: zwei getrennte Silizium-Bausteine, verbunden über die Platine.

Parallel vs. seriell

Parallele Übertragung:

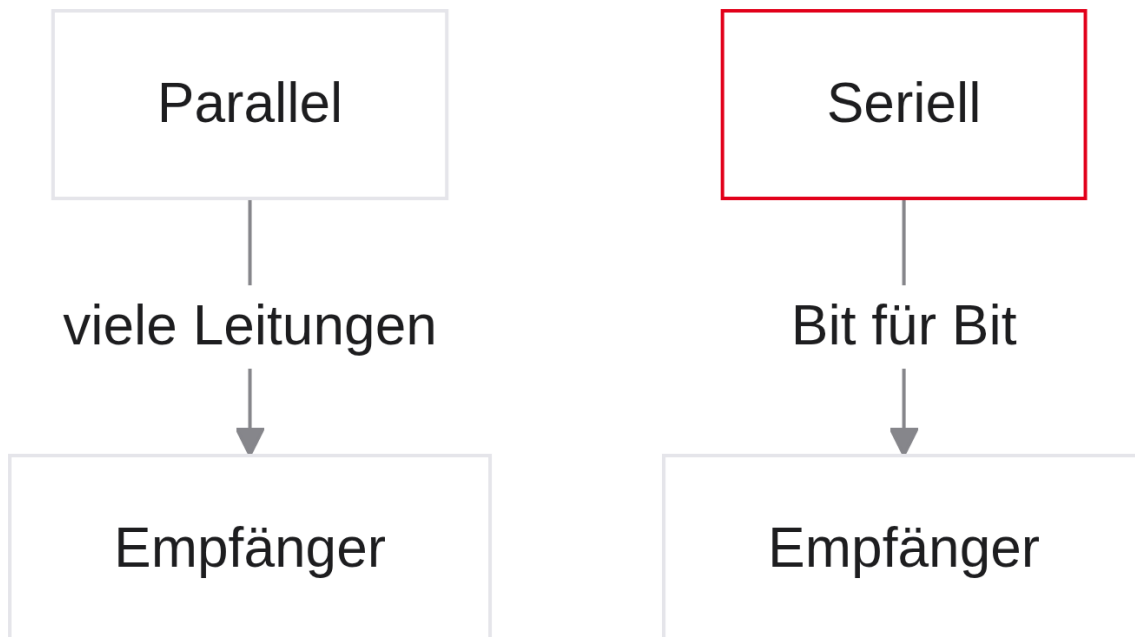
- Viele Leitungen (z. B. 32) — ein Wort in einem Takt
- So arbeiten typische interne CPU–RAM-Pfade

Serielle Übertragung:

- Wenige Leitungen (oft eine Datenleitung)
- Bits nacheinander — mehr Takte, weniger Pins

Notizen

Intuitiv wirkt parallel schneller. Extern haben sich dennoch serielle Standards durchgesetzt (USB, PCIe, On-Board-Busse). Warum?

Parallel vs. seriell — Schema**Notizen**

Parallel: Breite. Seriell: Sequenz auf wenigen Leitungen.

Warum sich Seriell durchgesetzt hat

- Kosten und Platz: wenige Pins am Gehäuse sind günstiger
- Crosstalk: viele parallele Bahnen stören sich gegenseitig
- Clock Skew: bei hohen Frequenzen kommen Bits nicht mehr gleichzeitig an
- Seriell kann hohe Symbolraten auf wenigen Leitungen erreichen
- „Seriell ist schneller“ gilt nicht pauschal: Pins, Takt und Nutzdatenrate getrennt betrachten
- Mehrere serielle Lanes (USB, PCIe) sind ein anderes Modell als UART/SPI/I2C

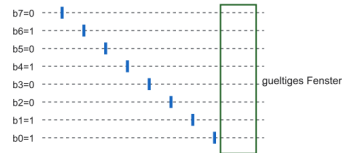
Notizen

Bei hohen Frequenzen wirken parallele Bahnen problematisch (Störung, Laufzeitunterschiede). Seriell braucht mehr Takte, erlaubt aber oft höhere Bitraten am Ende.

Das Schieberegister

0x53 = 0101 0011 in MSB-Schreibweise, UART sendet d0 zuerst.

Parallel, b7 links bis b0 rechts. Ankunft versetzt, ein gemeinsames Fenster.



Serielle Folge, LSB zuerst:



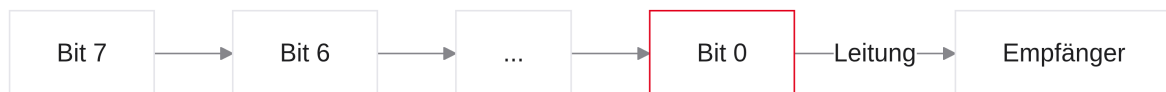
Schieberegister, vereinfachtes Links-Modell nur fuer die sichtbaren Bits. UART schiebt faktisch LSB hinaus.

Sender		Leitung	Empfänger
Start	0101 0011	Leitung leer	
nach d0,d1	0101 00..	1 dann 1	..000011
nach 8 Bit		Frame fertig	0101 0011

Seriell besetzt Laufzeit nicht. USB/PCIe sind andere Kodierungen, kein On-Board-UART.

Abbildung 2: 0x53 steht als 0101 0011 und geht als UART-Folge 1 1 0 0 1 0 1 0 hinaus.

- Wandlung: parallele CPU-Daten → serieller Bitstrom
- Kern: Schieberegister (Kette von D-Flip-Flops, MCT1)
- CPU schreibt parallel; Kommunikations-Hardware schiebt Bit für Bit auf die Leitung
- Empfänger schiebt Bits wieder zu einem Wort zusammen



Notizen

UART-, SPI- und I2C-Controller basieren im Kern auf solchen Schieberegistern — parallel rein, seriell raus und umgekehrt.

Synchron vs. asynchron

Synchrone Kommunikation:

- Eigene Taktleitung (Clock, z. B. SCK)
- Empfänger tastet bei klaren Taktflanken ab — robust und schnell

Asynchrone Kommunikation:

- Keine Taktleitung — nur Daten
- Beide Seiten brauchen vereinbarte Bitzeiten (Baudrate)

Notizen

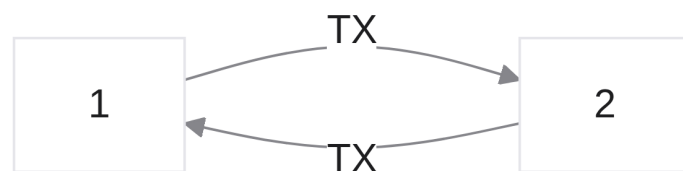
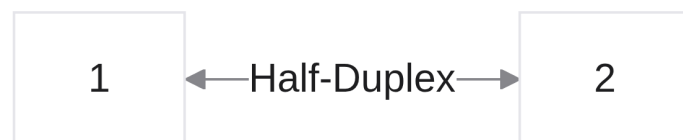
Ohne Takt: Wie unterscheiden Sie drei Nullen von einer langen Null? Synchron: der Sender gibt den Schlag vor. Asynchron: gemeinsame „Stoppuhren“.

Folie

11 / 80

Simplex, Half-Duplex, Full-Duplex

- Simplex: nur eine Richtung (z. B. GPS sendet Positionen)
- Half-Duplex: beide Richtungen, aber nicht gleichzeitig (ein Medium)
- Full-Duplex: getrennte Pfade — gleichzeitig senden und empfangen



Notizen

Wahlkriterium: Bandbreite vs. Pinzahl. Gelegentliche Sensorwerte → Half-Duplex oft genug; hohe Last → Full-Duplex.

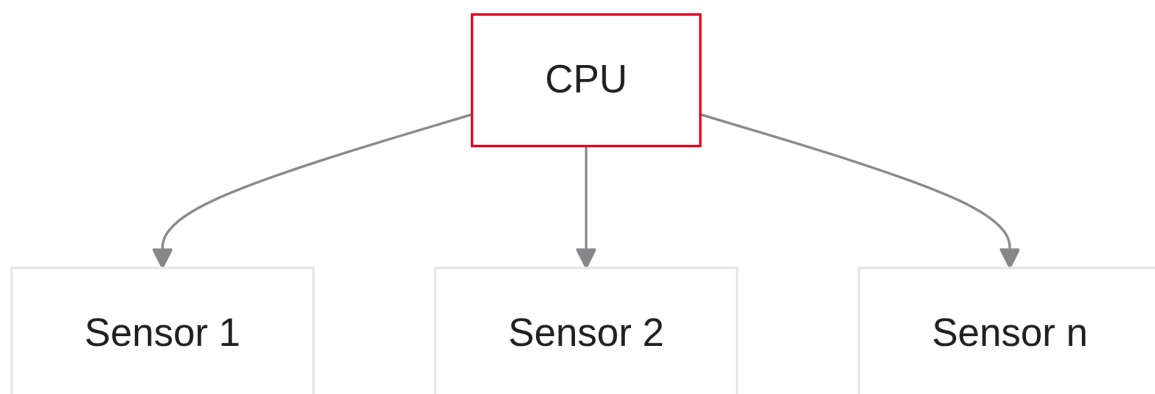
Die Leitfrage des Tages

- Punkt-zu-Punkt ist einfach
- Moderne Platinen: viele Sensoren/Chips an einer CPU
- Nicht für jeden Teilnehmer eigene Leitungsbündel (Pin-Mangel)
- Wie teilen sich viele Geräte wenige Pins, ohne Chaos und Kurzschluss?

Notizen

Netzwerk auf der Platine: Wer spricht wann? Wie vermeiden wir gleichzeitiges Treiben derselben Leitung?

Leitfrage — Schema



Notizen

Viele Teilnehmer, wenige Pins — das treibt UART vs. SPI vs. I2C.

Agenda: Die „Big 3“

Industrie-Standards für On-Board-Kommunikation:

1. UART — asynchron, Punkt-zu-Punkt
2. SPI — synchron, sehr schnell (Displays, Flash)
3. I2C — synchron, Netzwerk auf zwei Leitungen

	UART	SPI	I2C
Signale ohne GND	2	4, plus weitere CS	2
Takt	asynchron	synchron	synchron
Duplex	Full	Full im 4-Signal-Modell	Half
Grundtempo	Baud plus Framing	SCK aus dem Timingbudget	100/400 kHz als Fokus

Notizen

Alle drei finden Sie in typischen MCU-Datenblättern. Heute: wann welches Protokoll — Start mit UART.

Lernziele Block 5

- UART, SPI und I2C nach Pins, Takt und Topologie einordnen
- UART-Frame (Start/Stop/Parity) und Baudrate erklären
- SPI-Modi (CPOL/CPHA) und Chip-Select begründen
- I2C-Transaktion inkl. ACK/NACK und Repeated Start nachvollziehen
- Single-Ended vs. differenziell und EMI-Grundlagen einordnen

Notizen

Harris & Harris sowie Datenblatt-Kapitel zu Serial I/O — Protokollwahl ist Systemdesign.

Was ist UART?

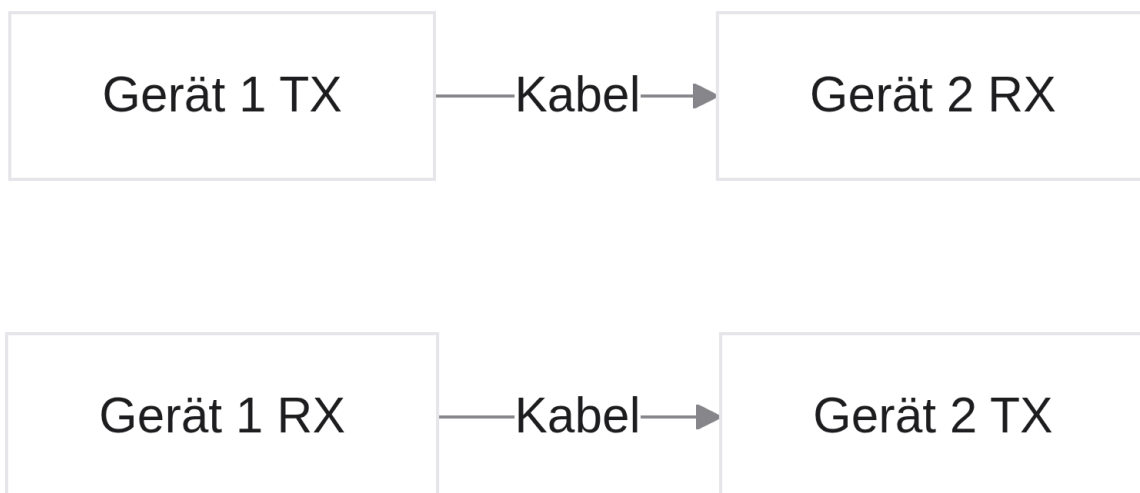
- UART: Universal Asynchronous Receiver-Transmitter
- Eher Schnittstelle/Direktverbindung als Multi-Drop-Bus
- Asynchron: kein Taktkabel
- Der UART-Frame ist nicht die elektrische Schnittstelle
- RS-232 nutzt andere Pegel; ein 0/3,3-V-Pin darf daran nicht direkt
- Heute oft: `Debug/print f` über einen Pegelwandler oder USB-UART

Notizen

Mindestens ein UART-Controller ist in MCUs üblich. Terminal-Ausgabe läuft typischerweise über UART-Hardware.

Punkt-zu-Punkt

- Typisch genau zwei Teilnehmer
- TX (Transmit) und RX (Receive)
- Kreuzverkabelung: TX1 → RX2, TX2 → RX1
- Direktes 0/3,3-V-Modell: gemeinsamer Bezug zwingend, keine galvanische Trennung
- Full-Duplex auf zwei Datenleitungen; GND, Versorgung und RTS/CTS zählen extra



Notizen

Einfache Verkabelung, gleichzeitiges Senden möglich — aber keine dritte Station am selben Link.

Die Asynchronität

- Kein Taktkabel
- Problem: Ist HIGH eine Eins oder fünf Einsen hintereinander?
- Lösung: vereinbarte Bitdauer — gleiche Baudrate auf beiden Seiten

Notizen

Ohne Absprache der Bitzeit liest der Empfänger Müll. Die Baudrate ist die gemeinsame Stoppuhr.

Die Baudrate

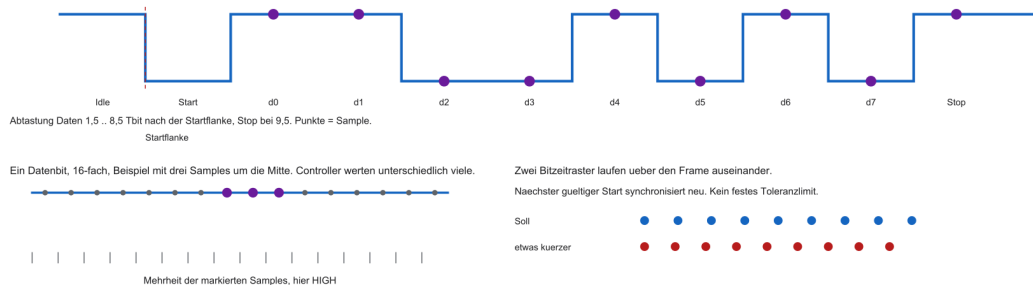
- Baud: Symbole pro Sekunde; hier typisch 1 Bit pro Symbol → bps
- Klassiker: 9600 oder 115200 Baud
- Bei 115200 Baud: Bitdauer $\approx 1/115200 \approx 8,68 \mu s$
- Prescaler/Baud-Generator aus Systemtakt in C konfigurieren

Notizen

Sender und Empfänger müssen dieselbe Baudrate nutzen — sonst wirre Zeichen. 115200 ist heute oft Standard für Konsolen.

Der UART-Frame

UART 8N1, 115200 Baud, Byte 0x53, LSB first, Ideales Modell.
 $T_{bit} = 1/115200 = 8,681 \text{ us}$. $T_{frame} = 10 T_{bit} = 86,806 \text{ us}$. Even Parity von 0x53 ist 0.



0x53 hat vier Einsen. Even-Parity-Bit = 0. Ein einzelner Kipper faellt auf, zwei Kipper koennen bleiben.

Stop falsch = Framing-Fehler. Parity falsch = eigener Status. Beides ist noch kein Paketprotokoll.

Optional 8E1: 11 Symbole, Datenanteil 8/11 etwa 72,7 Prozent. Leitungsrate bleibt 115200 Baud.

Direkt 0/3,3 V nur bei gemeinsamem Bezug. Ein UART-Pin vertraegt deshalb noch keine RS-232-Pegel.

Abbildung 3: 8N1 bei 115200 Baud: etwa 8,681 μs je Bit und 86,806 μs je Frame.

- Idle: Leitung typisch HIGH
- Frame-Aufbau:
 - Start-Bit: LOW
 - Daten: meist 8 Bit (oft LSB first)
 - Parity: optional
 - Stop-Bit(s): HIGH

Abschnitt	Pegel / Inhalt
Idle	HIGH
Start	LOW
Data	8 Bits
Stop	HIGH

Notizen

Jedes Byte wird eingerahmt. Das Start-Bit markiert den Paketbeginn im Bitstrom.

Das Start-Bit

- Start-Bit (LOW): Synchronisations-Trigger
- Fallende Flanke Idle → Start startet die Baud-Uhr des Empfängers
- Abtastung typisch nahe der Bitmitte (erstes Datenbit nach $\approx 1,5$ Bitzeiten)
- Danach Abtastung im Raster von 1 Bitzeit; nach dem Frame Uhr stoppen

Notizen

Die fallende Flanke ist der Startschuss. Abtastung in der Bitmitte gibt dem Signal Zeit zur Stabilisierung.

Oversampling

- Empfänger tastet oft mehrfach pro Bit ab (z. B. 16×)
- Mehrheitsentscheid um die Bitmitte filtert kurze Spikes
- Erhöht Robustheit bei gestörten Leitungen

Notizen

Ein einzelner Samplepunkt wäre anfällig für Störimpulse. Oversampling mittelt/wählt Mehrheiten in der kritischen Zone.

Clock Drift und Stop-Bit

- Quarze weichen ab (Temperatur, Toleranz) → Clock Drift
- Lange ununterbrochene Ströme würden Abtastpunkte verschieben
- Start-Bit kalibriert neu; Stop-Bit (HIGH) stellt Idle wieder her
- Nächstes Start-Bit erzeugt wieder eine klare fallende Flanke

Notizen

Stop-Bits sind kein Luxus: Sie begrenzen Drift-Akkumulation auf Frame-Länge und sichern die nächste Synchronisation.

Das Parity-Bit

- Optional: Even oder Odd Parity über die Datenbits
- Empfänger prüft; bei Fehler oft Status/Interrupt (Framing/Parity Error)

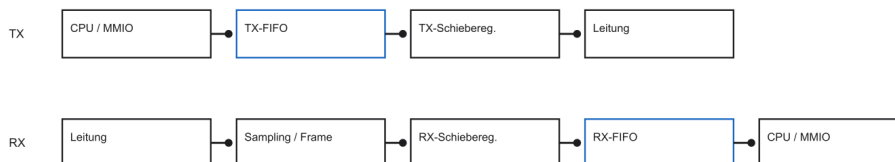
Daten (Beispiel)	Einsen	Even	Odd
1010 0001	3	1	0
0000 0000	0	0	1
0x53 = 0101 0011	4	0	1

Notizen

Parity erkennt ungerade Bitfehler; doppelte Kipper können unbemerkt bleiben. Für hohe Sicherheit: stärkere Prüfsummen in Software.

FIFOs im UART

Getrennte TX- und RX-Pfade. FIFO und Schieberegister sind zwei Stufen.



115200 Baud, 8N1: höchstens 11520 Byte/s, Datenanteil 80 Prozent.

RX-FIFO 16 Plätze, Watermark 8. Acht Bytes etwa 694 µs Spielraum, keine IRQ-Latenz.

Kommen weniger als acht Bytes, braucht es Timeout oder Idle. Overflow ist ein eigener Fehler.

Platz im TX-Puffer heisst nicht: letztes Stop-Bit ist schon auf der Leitung.

Fuer RS-485-Richtung zaehlt das gesendete Stop-Bit, nicht nur das leere FIFO.

Abbildung 4: Höchstens 11520 Byte/s. Acht freie RX-Plätze sind etwa 694 µs Spielraum.

- Bei 115200 Baud und 8N1 (10 Bit/Frame): grob alle $\approx 87 \mu s$ ein Byte
- Interrupt pro Byte belastet die CPU stark
- Lösung: RX/TX-FIFOs (z. B. 16–64 Bytes), Interrupt bei Watermark



Notizen

Erinnerung Block 1: FIFOs entkoppeln. Die CPU holt Burst-weise Bytes aus MMIO, statt jedes Zeichen einzeln zu bedienen.

Folie

26 / 80

Nachteile von UART

- Keine Geräteadressen → schlecht skalierbar (kein Multi-Drop)
- Baudrate muss auf beiden Seiten übereinstimmen (starr konfiguriert)
- Framing-Overhead (Start/Stop/Parity) und asynchrone Grenzen
- Für Displays/Kameras/viele Sensoren meist ungeeignet

Notizen

UART eignet sich hervorragend für Debug und einfache Module — nicht als Hochleistungs-Netzwerkbus.

Folie

27 / 80

Zusammenfassung UART

Eigenschaft	UART
Leitungen	TX, RX (+ GND)
Takt	asynchron (Baudrate)
Duplex	Full-Duplex
Topologie	Point-to-Point (2 Teilnehmer)
Overhead	Start/Stop/(Parity)
Einsatz	Terminal, GPS, Bluetooth-Module

Notizen

Merksatz: UART = asynchron, 2 Leitungen, 2 Teilnehmer. Als Nächstes: synchron mit Taktleitung — SPI.

Aufgabe — UART-Frame

Dekodieren Sie das Datenbyte. Idle ist HIGH, dann folgen Start, acht Datenbits LSB first und Stop.

- Sichtbare Datenbits nach dem Start: 1, 1, 0, 0, 1, 0, 1, 0
- Welche schriftliche MSB-Darstellung hat das Byte?
- Even Parity: welches Parity-Bit entsteht?

Notizen

Lösung: die Folge ist d0 bis d7, also 0x53 = 0101 0011. Vier Einsen, Even Parity = 0. Ein einzelner Bitfehler wäre erkennbar, zwei Bitfehler nicht sicher. Framing- und Parity-Fehler sind getrennte Statusbits.

Was ist SPI?

- SPI: Serial Peripheral Interface (u. a. Motorola, 1980er)
- Synchron: eigene Taktleitung — kein Baudraten-Drift wie bei UART
- Full-Duplex, oft sehr hohe SCK-Taktraten (einige zehn MHz möglich; Bitrate $\approx$ SCK bei 1 Bit/Takt)
- Wenig Framing-Overhead: Bits folgen dem Takt

Notizen

Der Controller gibt den Takt vor. Die Targets folgen dem Schlag. Eine gemeinsame Taktleitung verhindert die UART-Bauddrift, nicht jede beliebig hohe Frequenz.

Die Synchronität (SCK)

- Taktleitung: SCK (Serial Clock)
- Empfänger braucht keinen eigenen Baud-Generator
- Typisch: bei definierter Flanke Datenleitung abtasten
- Deterministisch, unempfindlich gegen Quarz-Abweichungen zwischen den Chips

Phase	Bedeutung
SCK-Flanke	Abtast-/Verschiebezeitpunkt
Datenleitung	gültiges Bit im vereinbarten Fenster

Notizen

Eine gemeinsame Uhr für Controller und Target. Die Uhren laufen nicht wie bei UART auseinander; das Timingbudget beider Datenblätter bleibt.

Die 4-Draht-Architektur

Klassisch 4 Signale (+ GND):

Signal	Richtung	Zweck
SCK	Master → Slave	Takt
MOSI	Master → Slave	Daten zum Slave
MISO	Slave → Master	Daten zum Master
CS / SS	Master → Slave	Slave auswählen (meist Active-Low)

Notizen

MOSI des Controllers trifft den Dateneingang des Targets. SDO und SDI gehören immer zu einem Baustein: Controller-SDO ist Target-SDI. Gleichnamig verbinden ist keine Regel.

Controller und Target

Ältere Datenblätter sagen Master und Slave. Hier: Controller und Target. MOSI und MISO bleiben die Signalnamen.

- Controller (meist die MCU): erzeugt SCK, steuert CS, startet Transfers
- Target: antwortet nur, wenn selektiert und getaktet
- Ein Controller: ein aktiver Taktgeber in diesem Grundmodell
- Das Target sendet nicht unaufgefordert

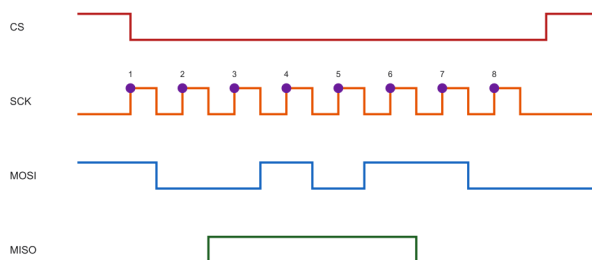
Notizen

Keine Buskollision durch unaufgefordertes Senden: der Master saugt/schiebt Daten aktiv.

Full-Duplex: Ringtausch

SPI Mode 0, MSB first. Controller 0x96, Target 0x3C. Aktives CS ist LOW.

Sample an der steigenden Flanke, Datenwechsel an der fallenden. Gemeinsamer Bezug gehoert dazu.



Gefuelter Punkt = Sample. Datenwechsel liegt auf der anderen Flanke.

Register, Linksverschieben und Einschieben: Start 0x96 / 0x3C

nach 4 Takten 0x63 / 0xC9, nach 8 Takten 0x3C / 0x96.

Lesen braucht oft ein Dummy-Byte. Volle FIFO beweist kein fertiges CS-Hold.

Full-Duplex gilt fuer dieses 4-Signal-Modell. 3-Draht, Dual und Quad sind Erweiterungen.

CS-Setup vor der ersten Sample-Flanke, CS-Hold nach dem letzten Bit.

Abbildung 5: Mode 0: 0x96 und 0x3C. Nach vier Takten 0x63 und 0xC9.

- Pro Takt: Bit auf MOSI hinaus und Bit auf MISO herein
- Nach n Takten: Schieberegister getauscht
- Auch zum reinen Lesen: Dummy-Bytes senden, damit SCK läuft



Notizen

SPI ist immer Tausch. Schreiben erzeugt Empfangsbytes (oft ignorieren); Lesen braucht Dummy-Writes.

Folie

34 / 80

Chip Select (CS)

- SCK/MOSI/MISO parallel an mehrere Slaves möglich
- CS wählt den aktiven Slave (meist Active-Low)
- Andere Slaves: Bus ignorieren, MISO hochohmig

Notizen

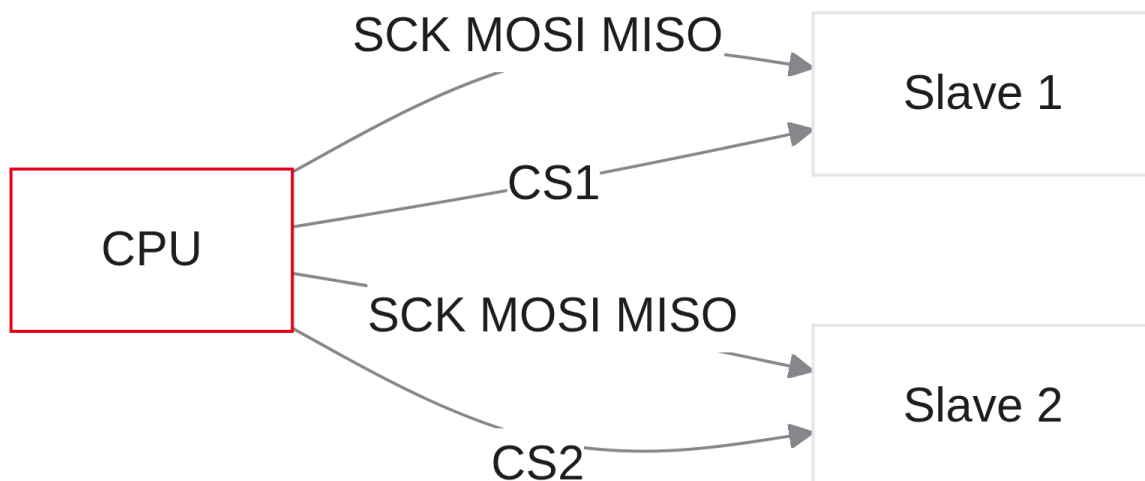
CS ist das „Scheinwerferlicht“: nur der selektierte Teilnehmer treibt MISO.

Folie

35 / 80

Star-Routing

- Jeder Slave: eigenes CS → eigener GPIO am Master
- 10 Slaves $\approx$ 3 Busleitungen + 10 CS = viele Pins
- SPI skaliert bei vielen Teilnehmern pinseitig schlecht

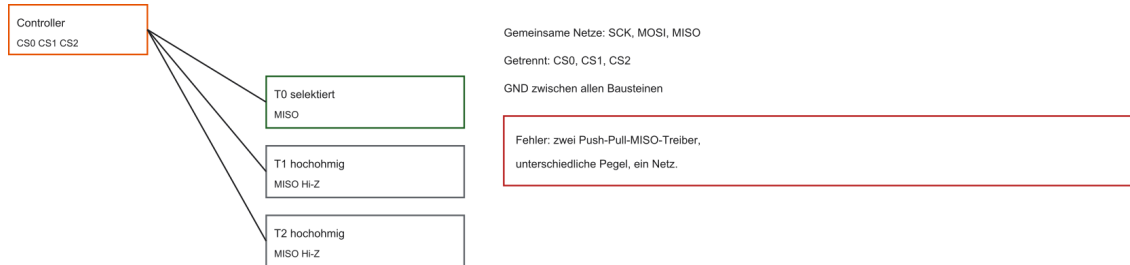


Notizen

Geschwindigkeit hoch, Pinverbrauch ebenfalls — Alternative: Daisy-Chain oder anderer Bus (I2C).

Daisy-Chaining

Stern: SCK, MOSI, MISO gemeinsam, drei CS, zusätzlich GND. Nur ein MISO-Treiber aktiv.



Daisy-Chain nur wenn das Gerät sie vorsieht. Start 00/00/00, Controller sendet 0x33, 0x22, 0x11.

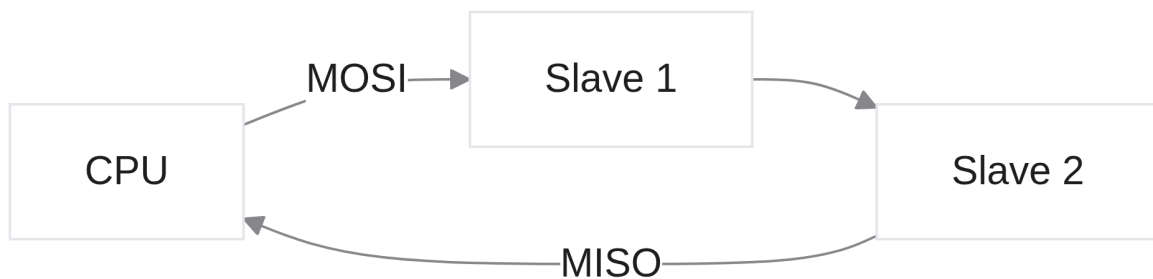
Nach 8 Takten 33/00/00, nach 16: 22/33/00, nach 24: 11/22/33. Latch ist ein extra Ereignis.

Controller-SDO gehoert an Target-SDI. Gleicher Name heisst nicht gleiche Leitung.

Ein Sensor mit Kommando/Antwort ist keine Daisy-Chain und hat im Grundmodell kein ACK.

Abbildung 6: Nur geeignete Bausteine. Nach 8, 16 und 24 Takten stehen 0x33, dann 0x22, dann 0x11 vorn.

- Ein CS für mehrere Schieberegister in Serie
- MOSI → Slave1 → ... → zurück MISO
- CPU schiebt lange Bitkette; Daten wandern durch die Kette
- Typisch bei LED-Ketten / Schieberegister-Arrays



Notizen

CS-Pins sparen, aber Updates betreffen die ganze Kette — für einzelne Slaves oft ineffizient.

Push-Pull und Geschwindigkeit

- SPI-Treiber typisch Push-Pull: aktiv HIGH und aktiv LOW
- Steile Flanken, schnelles Umladen der Leitungskapazität
- Ermöglicht hohe SCK-Frequenzen (je nach Layout und Slave)

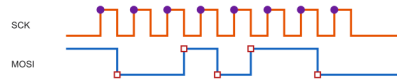
Notizen

Im Kontrast dazu später I2C mit Open-Drain und Pull-ups — robuster für Multi-Drop, aber langsamer.

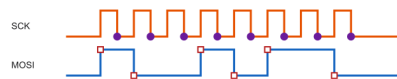
CPOL und CPHA

Vier Modi, gleiche MOSI-Folge 0x96. Leading geht vom Idle weg, Trailing kehrt zurück.

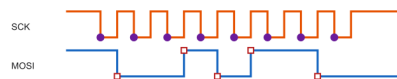
Mode 0 Idle LOW, Sample steigend



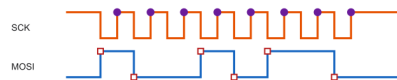
Mode 1 Idle LOW, Sample fallend



Mode 2 Idle HIGH, Sample fallend



Mode 3 Idle HIGH, Sample steigend



Punkt = Sample. Quadrat = Datenwechsel. Bei CPHA = 0 steht Bit 0 schon vor der ersten Sample-Flanke.

Bei CPHA = 1 bereitet die erste Flanke das Bit fuer die folgende Sample-Flanke.

Gleicher SCK verhindert Bauddrift, nicht beliebig hohe Frequenz. Setup, Hold, Clock-to-Output und Leitung begrenzen das Auge.

Maximale SCK-Frequenz aus beiden Datenblaettern und dem Layout, nicht aus dem Modus allein.

Abbildung 7: Vier Modi, gleiche Datenfolge. Sample und Datenwechsel sind verschiedene Flanken.

- Vier SPI-Modi (0–3): Kombination aus Ruhepegel und Abtastflanke
- CPOL: Idle LOW oder HIGH?
- CPHA: Abtastung bei erster oder zweiter Flanke?
- Immer Sensor-Datenblatt und Master-Register abgleichen — der Modus entscheidet über gültige Daten

Modus	CPOL	CPHA	Kurz
0	0	0	Idle LOW, Sample steigend
1	0	1	Idle LOW, Sample fallend
2	1	0	Idle HIGH, Sample fallend
3	1	1	Idle HIGH, Sample steigend

Notizen

Falscher Modus → Datenmüll trotz korrekter Verkabelung. Klassiker im Labor: alle vier Modi existieren; zwei Beispiele reichen nicht — Datenblatt lesen.

Nachteile von SPI

- Viele CS-Leitungen bei Star-Routing
- Nur physikalisches Bit-Schieben: keine Adressen, kein ACK in der Grundsicht
- Single-Master — Multi-Master schwierig
- Fehlende Slaves erkennt die Hardware nicht automatisch

Notizen

Software muss Plausibilität und Gerätepräsenz selbst prüfen.

Typische Einsatzgebiete

- Große Datenmengen, wenige Teilnehmer
- TFT-Displays, SD-Karten, externe Flash/EEPROM (auch Quad-SPI)
- Weniger ideal: dutzende langsame Sensoren (Pin-Overhead)

Notizen

SPI = Bandbreiten-Monster. Für viele kleine Sensoren kommt I2C.

Zusammenfassung SPI

Eigenschaft	SPI
Leitungen	SCK, MOSI, MISO, CS (+...)
Takt	synchron (Master)
Duplex	Full (Ringtausch)
Netz	Master/Slave, Star oder Daisy-Chain
Speed	hoch: SCK oft 10–50 MHz ($\approx$ Bitrate in Mbit/s bei 1 Bit/Takt)
Einsatz	Display, SD, Flash

Notizen

Als Nächstes: zwei Leitungen, Takt behalten, viele Adressen — I2C.

Folie

42 / 80

Aufgabe — SPI-Flanke

Mode 0, Idle LOW. Controller sendet 0×96 , Target sendet $0 \times 3C$, MSB first.

- Welche Flanke tastet ab, welche darf das nächste Bit setzen?
- Welcher Registerstand gilt nach vier Takten im Links-Schiebe-Modell?

Notizen

Lösung: Sample steigend, Wechsel fallend. Nach vier Takten 0×63 und $0 \times C9$, nach acht Takten $0 \times 3C$ und 0×96 . CPHA = 0 verlangt das erste Bit schon vor der ersten Sample-Flanke.

Folie

43 / 80

Was ist I2C?

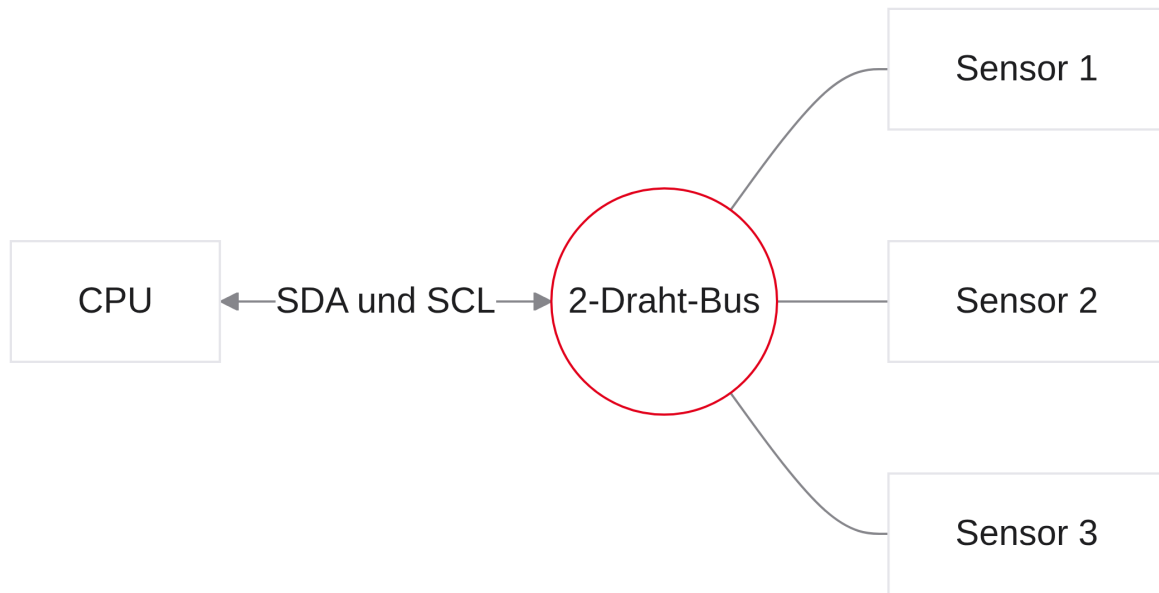
- I2C: Inter-Integrated Circuit (Philips/NXP, 1980er)
- Rückgrat vieler Sensor-Netzwerke auf der Platine
- Synchron: Taktleitung SCL (Serial Clock)
- Half-Duplex: eine Datenleitung SDA (Serial Data) — nicht gleichzeitig beide Richtungen
- Viele Bausteine an exakt zwei Signalen (+ GND)

Notizen

Auf Drohnen- oder Smartphone-Boards hängen Gyroskop, Barometer, Lade-IC und oft Touch am I2C — weil zwei Pins viele Geräte adressieren.

Ein echter Bus

- Kein SPI-Star-Routing mit vielen CS-Pins
- SCL und SDA als gemeinsame Busleitungen über die Platine
- Alle Teilnehmer parallel angeklemmt
- Dutzende Slaves an denselben zwei MCU-Pins möglich



Notizen

Echter Multi-Drop-Bus: gemeinsame Straße, geteilte Leitungen. Ohne CS: Adressierung in Software.

Adressierung statt CS-Kabel

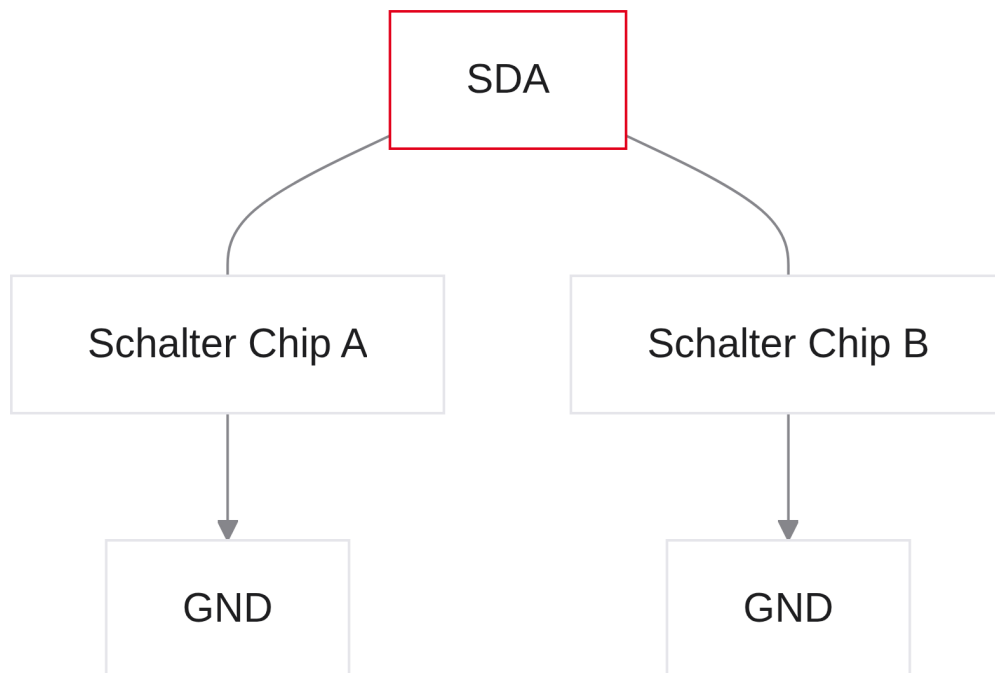
- Jeder Chip: feste Adresse (meist 7 Bit, z. B. 0×68 für MPU6050)
- Master sendet Adresse auf SDA
- Nur der angesprochene Slave antwortet; andere ignorieren
- 7 Bit $\Rightarrow$ theoretisch 128 Slots; praktisch weniger (reservierte Adressen); optional auch 10-Bit-Adressierung

Notizen

Hardware-Kabel durch Software-Adressen ersetzt — wie Namensaufruf im Klassenraum. Nicht jede 7-Bit-Kombination ist frei nutzbar; Details stehen im I2C-Spec und im Sensor-Datenblatt.

Open-Drain

- Problem: zwei Treiber gleichzeitig HIGH und LOW → bei Push-Pull Kurzschlussrisiko
- I2C: Open-Drain / Open-Collector
- Geräte können nur aktiv nach GND (LOW) ziehen
- HIGH: Leitung loslassen (hochohmig) — nicht aktiv „hochdrücken“

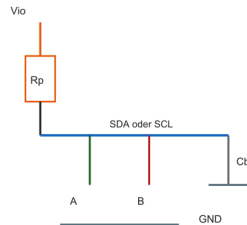


Notizen

Mehrere Geräte dürfen gleichzeitig LOW ziehen — Strom fließt nach Masse, ohne Push-Pull-Konflikt.

Pull-Up-Widerstände

Open-Drain-Lehrrechnung. $V_{IO} = 3,3 \text{ V}$, $C_b = 100 \text{ pF}$, $I_{OL} = 3 \text{ mA}$ bei $V_{OL,max} = 0,4 \text{ V}$.
Annahme, keine Messung. $t_r = 0,8473 \cdot R_p \cdot C_b$ fuer 30 bis 70 Prozent.



beide los: HIGH
nur A oder nur B: LOW
beide ziehen: LOW
HIGH heisst freigegeben, nicht aktiv getrieben.
Push-Pull HIGH gegen LOW ist ein Konflikt.
Open-Drain begrenzt diesen einen Fall, nicht jede Fehlverdrahtung.

Anstieg nach dem Loslassen, RC-Modell ab 0 V.

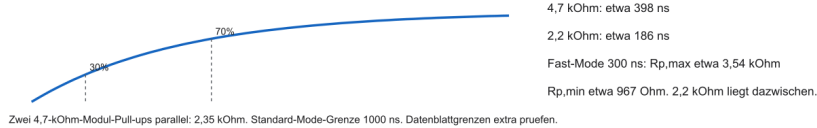


Abbildung 8: Lehrrechnung: 2,2 kΩ etwa 186 ns. Fast-Mode etwa 967 Ω bis 3,54 kΩ.

- Ohne aktives HIGH: je ein Pull-up an SDA und an SCL
- „Wenige kΩ“ ist keine Dimensionierung: Kapazität, Rise Time und Sinkstrom begrenzen R_p
- Je einer an SDA und SCL gegen V_{IO} (z. B. 3,3 V)
- Alle loslassen → Pull-up zieht HIGH (logisch 1)
- Ein Chip zieht → Leitung LOW (logisch 0)

Notizen

Ohne Pull-ups funktioniert I2C physikalisch nicht. HIGH heißt „niemand zieht“; LOW heißt „jemand zieht“.

Dominantes LOW (Wired-AND)

- LOW ist dominant, HIGH rezessiv
- Gleichzeitig 1 und 0 → Leitung wird 0
- Sender kann mitlesen: „Ich wollte 1, Bus ist 0“ → Überstimmt

Notizen

Wired-AND ist Grundlage für ACK und Arbitration bei Multi-Master.

I2C-Nachrichtenaufbau

Typische Transaktion:

1. START (SDA fällt bei SCL HIGH)
2. 7-Bit-Adresse
3. R/W-Bit (0 = Write, 1 = Read)
4. ACK vom Empfänger
5. Datenbyte(s) + ACK/NACK
6. STOP

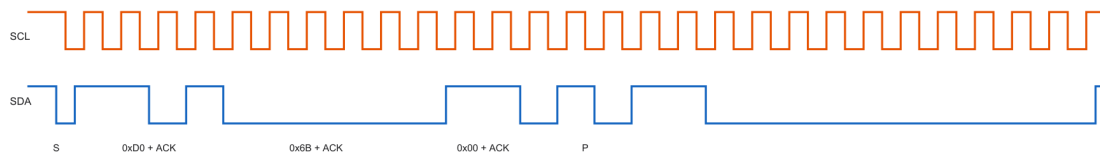
Feld	Bedeutung
START	Bus belegen, Slaves wecken
Adresse + R/W	Ziel und Richtung
ACK	Quittung
DATA	Nutzdaten

Notizen

START wirkt wie ein Gong: alle hören die Adresse; nur der Treffer bleibt aktiv.

ACK und NACK

I2C-Write, ein Controller, 7-Bit-Adresse 0x68, Write-Byte 0xD0, Register 0x6B, Daten 0x00.
SDA wechselt bei Daten nur während SCL LOW. Fallende SDA bei SCL HIGH = START, steigende = STOP.



Neunte Clock: Sender gibt SDA frei, Target zieht fuer ACK auf LOW.

Address-NACK: niemand zieht. Data-NACK: unerwartete Ablehnung.

Controller-NACK nach dem letzten gelesenen Byte ist der normale Abschluss, kein fehlendes Geraet.

0x68 << 1 | 0 = 0xD0, 0x68 << 1 | 1 = 0xD1. Eine API mit 7-Bit-Adresse schiebt intern. Nicht doppelt schieben.

Zwei Targets mit derselben Adresse trennt das Basisprotokoll nicht. AD0, Multiplexer oder getrennte Busse pruefen.

128 Bitmuster sind nicht 128 freie Geraete. Reservierte Adressen und optionale 10-Bit-Adressierung bleiben.

Ein ACK bestaetigt den Empfang des Bytes, nicht den physikalischen Messwert.

Abbildung 9: Write 0xD0, 0x6B, 0x00. Der letzte Read-NACK ist der normale Abschluss.

- Nach jedem Byte: 9. Takt für Quittung
- Sender lässt SDA los (Tendenz HIGH = NACK)
- Empfänger zieht SDA auf LOW = ACK
- Address-NACK: Gerät fehlt oder hört nicht. Data-NACK: unerwartete Ablehnung. Letzter Read-NACK: normaler Abschluss durch den Controller.

Notizen

Vorteil gegenüber SPI: die MCU erkennt fehlende Slaves auf Protokollebene.

Senden vs. Empfangen

Master Write:

- Adresse + R/W = 0 → ACK → Daten vom Master → ACK vom Slave

Master Read:

- Adresse + R/W = 1 → ACK → Slave treibt SDA, Master taktet weiter SCL
- Master sendet ACK/NACK für empfangene Bytes

Notizen

Half-Duplex: R/W wechselt die Richtung auf SDA; SCL bleibt beim Master (außer Stretching).

STOP-Condition

- Master beendet: bei SCL HIGH steigt SDA von LOW nach HIGH
- Signal: Bus frei für neue Transaktionen
- Slaves gehen zurück in Idle

Notizen

Klare Start- und Stop-Signaturen strukturieren den Verkehr auf dem gemeinsamen Medium.

Clock Stretching

Stretching: der Controller gibt SCL frei, das Target haelt LOW, die Leitung bleibt LOW.



Abtastung erst in der wirklichen HIGH-Phase, nicht beim Softwarebefehl HIGH.

Ob Stretching existiert und wie lange gewartet werden darf, steht im Controller, im Target und im Modell.

Arbitration, beide Write. A: 0x50 -> 0xA0. B: 0x68 -> 0xD0.

MSB beide 1. Danach sendet A eine 0, B eine 1. Die Leitung wird 0, B verliert und treibt nicht weiter.

A setzt 1 0 1 0 0 0 fort. B darf den Gewinn von A nicht mit einem STOP abbrechen.

Zustände: frei, START, Adresse, Daten, Ende. Ausgänge: NACK, Timeout, Arbitration Lost, Bus Error.

0x00 und 0xFF sind gueltige Bytes. Neun Clocks loesen nur festgehaltenes SDA bei steuerbarem SCL.

Ein Watchdog-Reset der CPU setzt einen extern versorgten Sensor nicht automatisch zurueck.

Abbildung 10: SCL bleibt LOW, solange das Target hält. Arbitration: 0xA0 gegen 0xD0.

- Zu schneller Master-Takt? Slave kann SCL auf LOW halten
- Master will HIGH, sieht aber LOW (Wired-AND) → wartet
- Slave lässt los → Transfer geht weiter

Notizen

Feature, das SPI so nicht hat: der Slave darf den Takt pausieren (z. B. interne Arbeit).

Multi-Master

- Mehrere Master auf demselben Bus möglich
- Beide können START generieren und Sensoren ansprechen
- Risiko: gleichzeitiger Buszugriff → Arbitration nötig

Notizen

I2C erlaubt Oligarchien mehrerer MCUs — SPI ist typisch Single-Master.

Arbitration

- Bei Kollision entscheidet dominantes LOW
- Wer 1 senden will, aber 0 liest, verliert und bricht ab
- Gewinner sendet weiter — Nachricht bleibt intakt

Notizen

Unblutige Kollisionsauflösung: Verlierer wechselt auf Empfang; Gewinner merkt oft nichts.

Nachteile von I2C

- Open-Drain + Pull-up: langsamere Flanken als Push-Pull
- Typisch Standard/Fast Mode: 100 kHz / 400 kHz (vs. SPI: SCK oft einige zehn MHz)
- Buskapazität begrenzt Teilnehmerzahl und Kabellänge

Notizen

Netzwerkfähigkeit kostet Bandbreite. Ideal für Sensoren; Displays eher SPI.

Zusammenfassung der Big 3

Merkmal	UART	SPI	I2C
Signale ohne GND	TX, RX	SCK, MOSI, MISO, CS	SDA, SCL
Takt	asynchron	Controller-Takt	Controller-Takt
Routing	Punkt-zu-Punkt	CS oder geeignete Daisy-Chain	adressierter Bus
Tempo	Baud mal Framing	Timingbudget beider Bausteine	100/400 kHz als Fokus
Treiber	Push-Pull, Pegel beachten	Push-Pull	Open-Drain, Pull-ups
Einsatz	Debug, Module	Display, Flash	Sensoren auf der Platine

Notizen

Auswahlkriterium: Pins vs. Bandbreite vs. Topologie.

Aufgabe — Adresse, ACK und Pull-up

- Geräteadresse 0×68 . Welches Byte geht bei Write, welches bei Read auf den Bus?
- Wer zieht SDA im neunten Takt eines Writes? Wer erzeugt den NACK nach dem letzten gelesenen Byte?
- Lehrannahme: $V_{IO} = 3,3 \text{ V}$, $C_B = 100 \text{ pF}$, 3 mA bei $0,4 \text{ V}$, $t_r = 0,8473 R_P C_B$. Liegt $2,2 \text{ k}\Omega$ im Fast-Mode-Fenster?

Notizen

Lösung: Write $0 \times D0$, Read $0 \times D1$. Das Target zieht ACK, der Controller erzeugt den letzten Read-NACK. $R_{p,min}$ etwa 967 Ohm , $R_{p,max}$ bei 300 ns etwa $3,54 \text{ kOhm}$. $2,2 \text{ kOhm}$ liegt in diesem Rechenmodell dazwischen. Zwei vorhandene $4,7 \text{ kOhm}$ parallel sind $2,35 \text{ kOhm}$. Datenblattgrenzen bleiben zu prüfen.

Grenzen von Single-Ended

- UART/SPI/I2C: Single-Ended — Pegel gegen gemeinsames GND
- Lange Kabel / getrennte Massepotentiale → Referenz driftet
- Empfänger sieht verfälschte Spannungen → Bitfehler

Notizen

On-Board (Zentimeter) funktioniert Single-Ended gut. Meterweite Maschinenverkabelung braucht andere Physik.

Elektromagnetische Interferenzen (EMI)

- Lange Leitungen wirken als Antennen
- Motoren/Wechselrichter induzieren Spikes
- Single-Ended: Spike kann 0 als 1 erscheinen lassen

Notizen

Werkshallen und Motorraum sind elektromagnetisch laut — On-Board-Busse sind dafür nicht ausgelegt.

Differentielle Übertragung

Single-Ended gegen Bezug, differentiell als Differenz. Common Mode bleibt begrenzt.

Single-Ended
Pegel gegen GND

Differentiell
 $V_{diff} = V_A - V_B$

$$V_{cm} = (V_A + V_B) / 2$$

Beispiel: $V_A = 2,5 \text{ V}$, $V_B = 0,5 \text{ V} \rightarrow V_{diff} = 2,0 \text{ V}$, $V_{cm} = 1,5 \text{ V}$

Dieselbe Störung $+0,8 \text{ V}$: V_{diff} bleibt $2,0 \text{ V}$, V_{cm} wird $2,3 \text{ V}$

Liegt $2,3 \text{ V}$ noch im erlaubten Bereich des gewählten Transceivers? Das Datenblatt entscheidet.

RS-485, Halbduplex
UART $\rightarrow$ Transceiver $\rightarrow$ A/B, Stubs kurz
Terminierung an beiden physikalischen Enden, Driver Enable steuert die Richtung

CAN separat
CAN-Controller plus CAN-Transceiver
Dominant/rezessiv. Rezessiv ist nicht zwingend invertiert.

Twisted Pair naehert gleiche Einkopplung an. Unsymmetrie und reale Empfänger bleiben.

Schichten: UART-Frame, dann RS-485-Transceiver, dann Leitung. Modbus liegt darüber.

RS-485 liefert keine CAN-Arbitration. Masse, Schutz und Isolation bleiben Systemfragen.

Mehrere hundert Meter sind keine beliebige Bitrate. Reichweite und Geschwindigkeit zusammen nennen.

Abbildung 11: $2,5 \text{ V}$ und $0,5 \text{ V}$: Differenz 2 V , Common Mode $1,5 \text{ V}$. $+0,8 \text{ V}$ hebt nur den Common Mode.

- Signal auf zwei Leitungen: normal und invertiert
- Empfänger misst Differenz $V_A - V_B$, nicht gegen GND
- Gleiche Störung auf beiden Leitungen ändert die Differenz nicht
- Der Common-Mode-Bereich des Empfängers bleibt endlich; große Potentialunterschiede können weiter stören oder schädigen



Notizen

Logik aus dem Vorzeichen/der Differenz der beiden Adern — nicht aus absolutem Pegel gegen GND.

Twisted Pair

- Adern eng verdreht
- Gleichtaktstörung hebt beide Adern ähnlich an
- Differenz bleibt: $(V_A + \Delta) - (V_B + \Delta) = V_A - V_B$

Notizen

Deshalb Ethernet und viele Feldbusse: Twisted Pair. Störungen heben sich in der Differenz auf.

RS-485

- Physikalische Schicht für lange Strecken (oft mit UART-Logik)
- Transceiver: Single-Ended TX/RX → differenziell A/B
- Reichweite und Bitrate gelten gemeinsam; „einige hundert Meter“ ist keine beliebige Geschwindigkeit
- Häufig mit Protokollen wie Modbus

Notizen

Sie programmieren weiter UART-ähnlich; der Transceiver macht die robuste Leitung.

CAN-Bus

- Controller Area Network (Bosch) — Standard im Fahrzeug
- Differenziell (CAN_H / CAN_L)
- Multi-Drop mit Arbitration (dominant/rezessiv)
- Verbindet Steuergeräte über ein Twisted-Pair

Notizen

CAN kombiniert Bus-Ideen (wie I2C) mit differentieller Robustheit für die Fahrzeugumgebung.

Warum im Labor I2C?

- MCT3-Fokus: Sensoren wenige Zentimeter neben der MCU
- Differentielle Transceiver: Platz, Strom, Kosten
- Für die „letzte Meile“ auf dem Board: I2C oft die passende Wahl

Notizen

Der Exkurs erklärt Feldverkabelung. Zurück aufs Board: I2C-Sensor ansteuern.

Aufgabe — Burst und Common Mode

- Bytes 10 00 E0 00 40 00 ab Register 0x3B, Bereich ± 2 g mit 16384 LSB/g. Welche Werte in g entstehen?
- $V_A = 2,5$ V, $V_B = 0,5$ V, danach +0,8 V auf beiden Leitungen. Was bleibt gleich, was ändert sich?

Notizen

Lösung: $X = 4096 = +0,25$ g, $Y = -8192 = -0,5$ g, $Z = 16384 = +1$ g. V_{diff} bleibt 2 V, V_{cm} steigt von 1,5 V auf 2,3 V. Ob 2,3 V erlaubt ist, steht im Transceiver-Datenblatt. Vertiefung: Multi-Controller, Daisy-Chain und die elektrische Auslegung.

Ausblick Labor: IMU per I2C

Geplantes Szenario:

- 3-Achsen-Beschleunigungssensor (IMU) am I2C-Bus
- Master: SCL erzeugen, Adresse senden
- Write: Sensor konfigurieren/aufwecken
- Read: Rohwerte X/Y/Z holen

Notizen

Details folgen im Übungsblatt. Ziel: eine vollständige I2C-Transaktion nachvollziehen.

MEMS kurz

- MEMS: mikromechanische Strukturen im Silizium
- Beschleunigung verbiegt Strukturen → Kapazität ändert sich
- Wandlung in Digitalwerte → Register im Sensor
- Diese Register lesen wir per I2C

Notizen

Mechanik im Chip-Maßstab liefert Zahlen; der Bus transportiert sie zur MCU.

Registerkarte des Sensors

- Sensor hat eigene Memory Map (Datenblatt!)
- Beispiele aus der MPU-6000/MPU-6050-Registerkarte, im Labor gegen das Datenblatt prüfen:
 - 0x6B: Power Management, Aufwecken ist bausteinspezifisch
 - 0x1C: Beschleunigungsbereich; ± 2 g entspricht im Datenblatt oft 16384 LSB/g
 - 0x3B bis 0x40: X, Y, Z je High/Low, zusammenhängend lesbar
- I2C: erst Geräteadresse, dann Registerzeiger, dann Daten

Notizen

Wie ein internes Regal: Adresse des Chips und Adresse des Fachs (Register).

C: I2C initialisieren

Beispielhafte Adressen und Bits (fiktiv — immer Datenblatt prüfen):

```
#define I2C_BASE 0x40010000u
#define I2C_CR1  (*(volatile uint32_t *) (I2C_BASE + 0x00))

void i2c_init(void) {
    /* Davor, plattformspezifisch: Taktfreigabe, Pin-Mux, Open-Dr
    I2C_CR1 |= (1u << 0); /* Enable - Bitlage nur im Lehrmodell
    I2C_CR1 |= (1u << 8); /* Controller-Modus - nur schematisch
}
```

- Register-Offsets und Bitpositionen sind plattformspezifisch
- Nicht 1:1 auf einen realen Chip übernehmen

Notizen

Wie Timer/PWM: Steuerung über MMIO. Enable-/Master-Bits hier nur zur Didaktik — im Labor die Referenz Ihres SoCs/Renode-Modells verwenden.

C: Bus-Frequenz

- Typisch 100 kHz oder 400 kHz
- Aus Systemtakt (z. B. 100 MHz) Teiler setzen

```
#define I2C_CLK_DIV (*(volatile uint32_t *) (I2C_BASE + 0x10))

void i2c_set_speed(void) {
    /* 100 MHz / 1000 = 100 kHz ist nur dieses Rechenmodell. */
    I2C_CLK_DIV = 1000;
}
```

Notizen

Prescaler allein ist keine universelle I2C-Formel. SCL-High, SCL-Low, Filter und die Anstiegszeit können getrennte Felder sein.

C: Write-Transaktion

Ablauf: START → Adresse+W → Reg → Daten → STOP

```
enum {
    I2C_OK = 0,
    I2C_NACK = 1,
    I2C_TIMEOUT = 2,
    I2C_ARB_LOST = 3,
    I2C_BUS_ERR = 4
};

static int i2c_fail(int st) {
    if (st != I2C_ARB_LOST && i2c_owns_bus())
        i2c_send_stop();
    return st;
}

int sensor_wakeup(void) {
    int st;
    i2c_send_start();
    i2c_send_byte(0xD0u);
    st = wait_phase();
    if (st != I2C_OK) return i2c_fail(st);
    i2c_send_byte(0x6Bu);
    st = wait_phase();
    if (st != I2C_OK) return i2c_fail(st);
    i2c_send_byte(0x00u);
    st = wait_phase();
    if (st != I2C_OK) return i2c_fail(st);
    i2c_send_stop();
    return I2C_OK;
}
```

Notizen

Ein NACK, ein Timeout, ein Arbitration-Verlust und ein Busfehler sind verschiedene Statuswerte. `wait_phase` begrenzt Start, Byte, Empfang, Stretching und Stop. STOP nur, solange dieser Controller den Bus besitzt.

C: Read mit Repeated Start

Burst-Read 0x3B bis 0x40. Kein STOP zwischen Zeiger und Read. Repeated Start haelt die Transaktion.



Controller bestaetigt Byte 1 bis 5 mit ACK und beendet Byte 6 mit NACK.

Bytes: 10 00 | E0 00 | 40 00

X = 0x1000 = 4096, Y = 0xE000 = -8192, Z = 0x4000 = 16384

+/-2 g und 16384 LSB/g: +0,25 g, -0,5 g, +1 g. Rohwert, Einheit und Bereich getrennt halten.

Tests: 0x0000 -> 0, 0x7FFF -> 32767, 0x8000 -> -32768, 0xFFFF -> -1

Unsigned zusammensetzen, dann als int16_t lesen. Kein Cast eines Bytepuffers auf int16_t*.

9 Bytes * 9 SCL = 81 Pulse. 810 us bei 100 kHz, 202,5 us bei 400 kHz, ohne Stretching.

Nutzdaten 48/81, etwa 59 Prozent. Ein einzelnes Byte ab 0x3B ist nur X-High.

Auto-Increment und ein gemeinsamer Snapshot sind Eigenschaften des Sensors, nicht von I2C allgemein.

Getrennte Transaktionen koennen drei Achsen aus verschiedenen Abtastungen mischen.

Abbildung 12: Sechs Bytes ab 0x3B. Das Beispiel ergibt +0,25 g, -0,5 g und +1 g.

Zuerst Registerzeiger setzen (Write), dann Repeated START und Read:

```
int read_accel_xyz(uint8_t out[6]) {
    int st;
    i2c_send_start();
    i2c_send_byte(0xD0u);
    st = wait_phase();
    if (st != I2C_OK) return i2c_fail(st);
    i2c_send_byte(0x3Bu);
    st = wait_phase();
    if (st != I2C_OK) return i2c_fail(st);
    i2c_send_start();
    i2c_send_byte(0xD1u);
    st = wait_phase();
    if (st != I2C_OK) return i2c_fail(st);
    for (int i = 0; i < 5; i++) {
        st = i2c_read_ack(&out[i]);
        if (st != I2C_OK) return i2c_fail(st);
    }
    st = i2c_read_nack(&out[5]);
    if (st != I2C_OK) return i2c_fail(st);
    i2c_send_stop();
    return I2C_OK;
}
```

Notizen

Klassisches Muster: Pointer setzen, dann lesen — oft ohne STOP dazwischen (Repeated Start).

Folie

74 / 80

Renode: virtueller I2C-Sensor

- Ein Registermodell kann ACK und Bursts nachbilden
- Es beweist keine realen Flanken, keine RC-Zeit und keine zyklusgenaue Controllerlatenz
- `Sensors.MPU6050` und `FeedSample` sind hier nicht als vorhandene API geprüft

Notizen

Simulator ersetzt die physische IMU für Treiberentwicklung — Datenblattverhalten nachbilden.

Folie

75 / 80

Platine beschreiben (`.repl`)

Im Repository liegt keine `.repl`. Der folgende Typname ist ein ungeprüftes Schema, keine Renode-API dieser Lehrveranstaltung:

```
# nicht ausgeführt, Typname nicht verifiziert
# sensor: Sensors.MPU6050 @ i2c0 0x68
```

- Erst die installierte Renode-Version und das echte Peripheriemodell prüfen
- Fehlt das Modell, bleibt der Treiber ein Lehrmodell mit explizitem Registervertrag

Notizen

Die `.repl`-Datei beschreibt die virtuelle Verdrahtung — Ihr „Lötkolben“ in Textform.

Stimuli im Startskript (. resc)

Schematisch:

```
# Schema, in dieser Version nicht ausgeführt
# machine LoadPlatformDescription @board.repl
# sysbus LoadELF @firmware.elf
# FeedSample ist kein geprüfter Befehl mit drei Fließkomma-Argumenten
```

- Virtuelle Registertransfers können Treiberfehler zeigen
- Sie beweisen keine Pull-up-Auslegung, keine RC-Flanke und keine Controllerlatenz

Notizen

Laboridee: eingespeiste Vektoren müssen nach korrektem I2C-Read in Ihrer Ausgabe erscheinen.

Endianness

- 16-Bit-Werte oft als High- dann Low-Byte (Big-Endian am Sensor)
- In C zusammensetzen:

```
int16_t decode_x(const uint8_t b[6], int st, int *ok) {
    uint16_t ux;
    if (st != I2C_OK) {
        *ok = 0;
        return 0;
    }
    ux = (uint16_t)((uint16_t)b[0] << 8) | b[1];
    *ok = 1;
    return (int16_t)ux; /* 0x1000 -> 4096 */
}
```

Notizen

Häufiger Fehler: den Puffer trotz Fehlerstatus dekodieren. High-Byte liegt bei `b[0]`. Der Cast auf `int16_t` übernimmt das Zweierkomplement, nachdem die Bytes unsigned zusammengesetzt wurden. `0x00` bis `0xFF` bleiben gültige Messbytes.

Polling vs. Interrupt

- Blockierendes Polling, Zustands-Polling und IRQ/DMA sind drei Implementierungen
- Ein I2C-IRQ ist auf RISC-V eine externe Quelle, bei einem PLIC die Claim-ID, keine eigene `cause`-Geräteerkennung
- Jede Warteschleife braucht eine Deadline. Ein CPU-Reset setzt den extern versorgten Sensor nicht zurück

Notizen

Erinnerung Block 3: Polling blockiert; Interrupts entlasten die CPU bei längeren Transfers.

Zusammenfassung Block 5

- Seriell ersetzt parallel extern: Pins, Crosstalk, Skew
- UART: asynchron, Punkt-zu-Punkt, Debug
- SPI: synchron, schnell, CS-intensiv
- I2C: adressierter Open-Drain-Bus auf zwei Leitungen
- Differenziell (RS-485, CAN): lange, gestörte Strecken
- Treiber: MMIO, Adresse|R/W, Registerzeiger, Bit-Shifting

Notizen

Damit kann Ihr SoC Module ansprechen, Sensoren lesen und Konsolen nutzen — Grundlage für vernetzte Embedded-Systeme.

Ausblick und Abschluss

- Übungsblatt: I2C in Renode Schritt für Schritt
- Ziel: erste virtuelle Beschleunigungswerte auslesen

Vielen Dank — viel Erfolg im Labor.

Notizen

Firmware-Entwicklung heißt: Protokollregeln in Registerzugriffe und Timing übersetzen. Viel Erfolg beim Experimentieren.